Movie Collection App

Architecture & Design Document

A comprehensive guide for software engineers

Document Overview

- **Purpose:** Technical architecture and design patterns for the Movie Collection App
- **Audience:** Software Engineers, Architects, Developers
- **Last Updated:** November 2025

Table of Contents

1. System Overview

Project Description

Movie Collection App is a cross-platform movie management application built with Flutter.

Core Purpose:

- Track and manage personal movie collections
- User authentication and profile management
- Wishlist and favorites functionality
- Integration with external movie APIs (e.g., TMDB)
- Data persistence and cloud sync

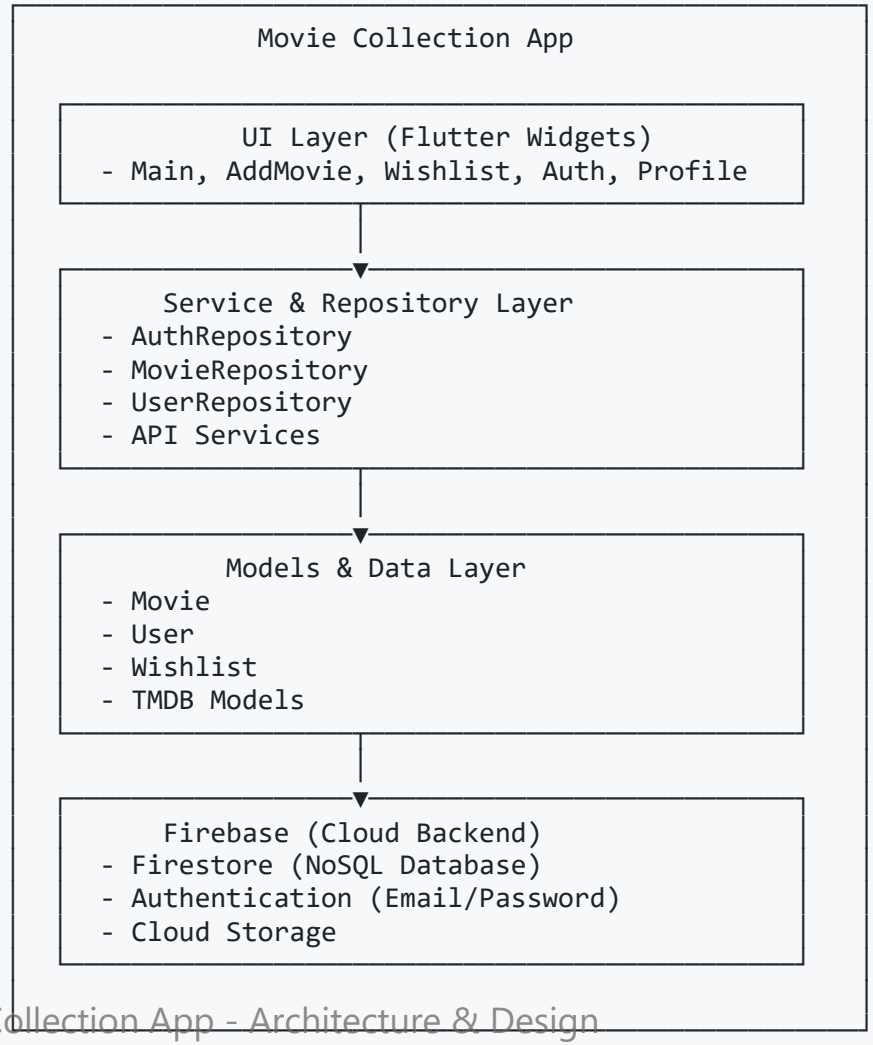
Technology Stack:

- **Frontend:** Flutter (Dart)
- **Backend:** Firebase (Firestore + Authentication)
- **Architecture:** Layered (Clean Architecture, MVC)
- **Minimum SDK:** Flutter 3.10.0+, Dart 3.0.0+

Key Features

Feature	Status	Priority
Movie Management	✅ Complete	Critical
Wishlist	✅ Complete	High
User Authentication	✅ Complete	Critical
Data Persistence	✅ Complete	Critical
API Key Setup	✅ Complete	Medium
External API Integration	✅ Complete	Medium
Profile Management	✅ Complete	Medium

System Architecture Diagram



2. Architecture Patterns

Layered Architecture Implementation

The application follows a layered architecture with separation of concerns:

```
lib/
├── models/              ← Data Models (Pure Dart)
├── repositories/        ← Data Access & Business Logic
├── services/            ← API & Utility Services
├── screens/             ← UI Components & Pages
├── core/                ← Constants, Error Handling, DI
├── config/              ← Configuration (API Keys, etc)
```

Layer Responsibilities

Models Layer:

- Pure data classes
- Serialization/deserialization
- Immutability where possible
- Examples: `Movie`, `User`, `Wishlist`, `TMDB Models`

Repositories Layer:

- Data access logic
- Firebase operations
- API integration
- Examples: `AuthRepository`, `MovieRepository`, `UserRepository`

Services Layer:

Dependency Flow

Unidirectional Dependency Flow:

```
Screens (UI)
    ↓
Services & Repositories
    ↓
Models
    ↓
Firebase & External APIs
```

Benefits:

- Clear separation of concerns
- Testable and maintainable code
- Scalable for new features

3. Data Models & Structure

Core Data Models

Movie Model

```
class Movie {
  final String id;
  final String title;
  final String overview;
  final String posterUrl;
  final DateTime releaseDate;
  final List<String> genres;
  final double rating;
  final bool isFavorite;
}
```

Responsibilities:

- Represent a single movie entry
- Store metadata and user preferences

User Model

```
class User {
  final String id;
  final String email;
  final String displayName;
  final String photoUrl;
  final List<String> wishlist;
}
```

Wishlist Model

```
class Wishlist {
  final String userId;
  final List<String> movieIds;
}
```

TMDB Models

- Used for external API integration
- Map TMDB API responses to app models

Firestore Data Structure

```
firestore/
├── users/
│   ├── {userId}/
│   │   ├── profile/              (User profile data)
│   │   ├── movies/               (User's movie collection)
│   │   │   └── {movieId}
│   │   ├── wishlist/             (Wishlist movies)
```

Collection Structure Benefits:

- Organized by user
- Subcollections for scalability
- Clear data ownership

4. Service & Repository Layer

Repository Architecture

Purpose: Centralize data access and business logic

Principles:

- Async operations for I/O
- Error handling
- Data validation

AuthRepository

```
class AuthRepository {
  Future<User?> signUp(String email, String password);
  Future<User?> signIn(String email, String password);
  Future<void> signOut();
  User? getCurrentUser();
}
```

MovieRepository

```
class MovieRepository {
  Future<void> addMovie(String userId, Movie movie);
  Future<List<Movie>> getMovies(String userId);
  Future<void> updateMovie(String userId, Movie movie);
  Future<void> deleteMovie(String userId, String movieId);
}
```

UserRepository

```
class UserRepository {
  Future<User> getUserProfile(String userId);
  Future<void> updateUserProfile(String userId, User user);
}
```

API Services

- TMDB API integration for movie data
- API key setup and management

5. UI/View Layer

Widget Architecture

Hierarchy:

```
MyApp (Root)
└── AuthWrapper (Authentication Gate)
    ├── APIKeySetupPage (if needed)
    └── MainScreen (if authenticated)
        ├── AppBar
        │   ├── Title
        │   └── Actions
        │       ├── AddMovie
        │       ├── Wishlist
        │       └── Profile
        └── Body
            ├── MovieList
            └── Wishlist
```

Main Navigation Screens

Screen	Purpose	Key Features
MainScreen	Dashboard	Movie list, add/view movies
AddMoviePage	Add new movie	Form, validation
WishlistPage	Wishlist management	Add/remove favorites
APIKeySetupPage	API key configuration	TMDB API setup
ProfilePage	User profile	Edit profile, sign out
AuthWrapper	Auth flow	Login/signup, state mgmt

Material Design Implementation

- Custom color scheme
- Responsive layouts
- Theming via `core/constants.dart`

6. Firebase Integration

Firebase Setup

- Firestore Database (NoSQL)
- Authentication (Email/Password)
- Cloud Storage

Initialization:

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(MyApp());
}
```

Authentication Flow

```
User Opens App
    ↓
AuthWrapper Checks Auth State
    ├─ Not Logged In
    │   └─ Show Login/Signup
    └─ Logged In
        └─ Show MainScreen
            └─ Load User's Data
                └─ Movies, Wishlist, Profile
```

Data Persistence Pattern

Creating/Adding Data:

```
Future<void> addMovie(String userId, Movie movie) async {
  await FirebaseFirestore.instance
    .collection('users').doc(userId)
    .collection('movies').doc(movie.id)
    .set(movie.toMap());
}
```

Reading Data:

```
Future<List<Movie>> getMovies(String userId) async {
  final snapshot = await FirebaseFirestore.instance
    .collection('users')
    .doc(userId)
    .collection('movies')
    .get();

  return snapshot.docs
    .map((doc) => Movie.fromMap(doc.data(), doc.id))
    .toList();
}
```

Updating Data:

```
Future<void> updateMovie(String userId, Movie movie) async {
  await FirebaseFirestore.instance
    .collection('users')
    .doc(userId)
    .collection('movies')
    .doc(movie.id)
    .update(movie.toMap());
}
```

Deleting Data:

```
Future<void> deleteMovie(String userId, String movieId) async {
  await FirebaseFirestore.instance
    .collection('users')
    .doc(userId)
    .collection('movies')
    .doc(movieId)
    .delete();
}
```

Firestore Security Rules

Best Practices:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId} {
      allow read, write: if request.auth.uid == userId;
      match /movies/{movie=**} {
        allow read, write: if request.auth.uid == userId;
      }
      match /wishlist/{wishlist=**} {
        allow read, write: if request.auth.uid == userId;
      }
    }
  }
}
```

7. Testing Strategy

Test Categories

Category	Purpose	Type
Unit Tests	Test individual functions/classes	Dart
Widget Tests	Test widget behavior	Flutter
Integration Tests	Test service + Firebase interaction	Flutter
Acceptance Tests	Business logic from user perspective	Flutter
Regression Tests	Prevent breaking changes	Dart

Test Organization

```
test/
├── unit/                    # Dart unit tests
├── widget/                  # Widget tests
├── integration/             # Integration tests
├── acceptance/              # Acceptance tests
├── regression/              # Regression tests
```

Running Tests

```
# Run all tests
flutter test

# Run specific test file
flutter test test/unit/movie_test.dart

# Run tests matching pattern
flutter test -k "movie"

# Run with coverage
flutter test --coverage
```

8. Deployment & Infrastructure

Build Configuration

pubspec.yaml Dependencies:

```yaml
dependencies:
  flutter:
    sdk: flutter
  firebase_core: ^4.1.0
  cloud_firestore: ^6.0.1
  firebase_auth: ^6.1.1
  http: ^1.2.0
  intl: ^0.20.2
```

Platform-Specific Considerations

Android Configuration

```
<!-- AndroidManifest.xml -->
<uses-permission android:name="android.permission.INTERNET" />
```

iOS Configuration

```
# ios/Podfile
platform :ios, '11.0'
```

Build & Release Process

Development Build:

```
flutter run
```

Release Build (Android):

```
flutter build apk --release
```

Release Build (iOS):

```
flutter build ipa --release
```

Web Deployment:

```
flutter build web --release
```

Environment Variables

Firebase Configuration:

```
// firebase_options.dart (auto-generated)
const FirebaseOptions currentPlatform =
  FirebaseOptions(
    apiKey: 'YOUR_API_KEY',
    appId: 'YOUR_APP_ID',
    projectId: 'YOUR_PROJECT_ID',
    storageBucket: 'YOUR_BUCKET',
    messagingSenderId: 'YOUR_SENDER_ID',
  );
```

Performance Optimization

- Efficient database queries
- Lazy load images
- Use ListView.builder for large lists
- Dispose controllers and streams

Monitoring & Debugging

- Flutter DevTools
- Firebase Console
- Crashlytics
- Analytics

Security Best Practices

- Never commit secrets
- Validate user input
- Use HTTPS
- Authenticate requests
- Firestore Rules
- Data encryption

Summary & Best Practices

Architecture Checklist

- ✅ Layered Architecture
- ✅ MVC Pattern
- ✅ Repository & Service Layer
- ✅ Firebase Integration
- ✅ Async Operations
- ✅ Error Handling
- ✅ Testing
- ✅ Documentation

Development Workflow

1. **Feature Development**
    - Create feature branch
    - Write tests first (TDD)
    - Implement feature
    - Verify all tests pass
    - Create pull request
2. **Code Review**
    - Check for architecture violations
    - Verify test coverage
    - Review documentation
    - Test on multiple devices

Common Patterns

Repository Method Pattern:

```
Future<T> methodName(
  String userId,
  // required parameters
) async {
  try {
    // Business logic
    return result;
  } catch (e) {
    debugPrint('Error: $e');
    rethrow;
  }
}
```

Widget State Pattern:

```
void initState() {
  super.initState();
  _loadData();
}

Future<void> _loadData() async {
  try {
    final data = await repository.getData();
    setState(() { _data = data; });
  } catch (e) {
    _showError(e);
  }
}
```

Further Reading

- **Flutter Docs:** https://flutter.dev/docs
- **Dart Docs:** https://dart.dev/guides
- **Firebase Docs:** https://firebase.google.com/docs
- **Clean Architecture:** https://researchgate.net/publication/303887935
- **MVC Pattern:** https://en.wikipedia.org/wiki/Model%E2%80%93view%E2%80%93controller

Contact & Support

- Review code documentation
- Check inline comments
- Consult design patterns
- Run tests for examples

Development Setup:

```
# Clone repository
git clone <repo-url>

# Install dependencies
flutter pub get

# Run app
flutter run

# Run tests
flutter test
```

End of Architecture & Design Document